

CHAPTER 7

TESTING

7.1 TESTING

Testing for the DepHNN Analyzer project was conducted using a structured methodology to ensure the system's accuracy, reliability, and robustness. This included unit testing, integration testing, system testing, and user acceptance testing (UAT), allowing early detection and correction of defects and ensuring seamless operation of all components. The primary objective was to validate that the system correctly performs EEG data upload, pre-processing, model inference, and results visualization.

Special emphasis was placed on model-specific testing to verify that the CNN + LSTM hybrid model accurately processes temporal EEG data, extracts relevant features, and produces reliable predictions. The system's performance and scalability were also evaluated by measuring prediction time, testing with large EEG datasets, and simulating multiple concurrent users to ensure efficiency and responsiveness in real-world scenarios.

Additionally, the user interface and security aspects were tested to guarantee an intuitive experience, responsive design across devices, and safe handling of sensitive EEG data. Overall, this comprehensive testing framework confirmed that the DepHNN Analyzer is functionally correct, robust, user-friendly, and ready for deployment for EEG-based depression detection.

7.2 TYPES OF TESTS

7.2.1 UNIT TESTING

Unit testing for the DepHNN Analyzer focused on verifying the smallest testable components of the system in isolation. Individual functions, methods, and modules were tested to ensure correct behaviour prior to integration. The unit tests covered user management, OTP generation, form validation, EEG preprocessing, and model loading, as summarized below:

User Model: Tests included creating the CustomUser model with valid data, handling missing required fields (e.g., username), and verifying password hashing functionality.

OTP Generation: Verification of generate_otp() producing a 6-digit number, correct OTP model creation and storage, and proper OTP expiry logic.

Django Forms: Validation of RegistrationForm for valid email, strong password, and unique username, and LoginForm validation for correct credentials.

MNE-Python Preprocessing Functions: Tests ensured load_edf_file() handles valid and invalid files, select_channels() extracts the 19 EEG channels correctly, resample_data() maintains 256 Hz frequency, apply_filter() applies the 0.5–45 Hz bandpass, run_ica() fits ICA components successfully, normalize_data() applies Z-score normalization, and window_data() generates 1-second windows with 50% overlap.

Keras Model Loading: Tests confirmed successful loading of the .keras model file and verified that model.predict() executes correctly with dummy input of shape (1, 19, 256).

These unit tests ensured that each core module and function of the DepHNN Analyzer operates as intended, providing a robust foundation for integration and system-level testing.

7.2.2 INTEGRATION TESTING

Integration testing for the DepHNN Analyzer project focused on verifying the interactions between different modules and ensuring that combined components work together as expected. This level of testing ensures that the system functions correctly when individual units are integrated, and that data flows seamlessly through different stages of the application. The integration tests covered user management, preprocessing, model prediction, and API endpoints, as outlined below:

User Registration Flow: Tests verified the submission of RegistrationForm leading to CustomUser creation and OTP generation, followed by OTP verification and user account activation.

User Login/Logout Flow: Tests confirmed successful login via LoginForm with session creation, proper handling of invalid credentials, and session termination upon logout. This ensures secure authentication and consistent user access control throughout the application.

Preprocessing to Prediction Flow: Tests ensured that the output of `views.preprocess_single_file()` feeds correctly into `model.predict()` and that data shapes remain consistent between preprocessing and model input.

API Endpoint Integration: Tests verified that the `/predict/` endpoint correctly processes uploaded `FormData`, executes preprocessing, invokes the model, and returns a JSON response. Additional tests confirmed proper error handling when no file is uploaded.

These integration tests confirmed that the DepHNN Analyzer modules interact correctly, preserving data integrity and ensuring a smooth, end-to-end workflow from user interaction to depression prediction.

7.2.3 FUNCTIONAL TESTING

Functional testing for the DepHNN Analyzer focused on verifying that each system feature performs according to its specifications, ensuring accurate and reliable operation from user interaction to EEG data analysis and prediction display. The tests covered user authentication, file upload and analysis, and result display, as summarized below:

User Authentication: Tests confirmed that new users can register, receive an OTP, and activate their account successfully. Registered users were verified to log in and log out correctly. The system was also tested to ensure that unauthenticated users cannot access secure endpoints such as `/predict/` and are redirected to the login page when attempting to access protected routes.

File Upload & Analysis: Tests verified that users can upload valid `.edf` EEG files and receive predictions, while the system correctly handles invalid file types (e.g., `.txt`, `.jpg`) and corrupted or malformed `.edf` files by displaying appropriate error messages.

Result Display: Tests ensured that the prediction labels ("Depressed" / "Healthy") and confidence scores are displayed accurately. The Chart.js donut chart correctly visualizes the proportion of healthy versus depressed windows, and detailed metrics including total windows, depressed windows, and healthy windows are rendered correctly.

These functional tests validated that the DepHNN Analyzer's core features operate reliably, delivering a seamless user experience and accurate EEG-based depression detection. The successful execution of each feature confirms that both backend processing modules and user

interface components communicate effectively without errors. This ensures that the system is ready for deployment and capable of supporting real-world clinical or research applications.

7.2.4 SYSTEM TESTING

System testing for the DepHNN Analyzer evaluated the fully integrated software to ensure it meets all specified requirements. This testing validated end-to-end functionality, performance, security, and user experience under realistic conditions. The tests encompassed end-to-end prediction, performance, security, error handling, and browser compatibility, as summarized below:

End-to-End Prediction: The full workflow—from registration and login to .edf file upload, analysis, result viewing, and logout—was executed multiple times with different datasets, including known healthy and depressed EEG files, to ensure consistent and accurate predictions.

Performance Testing: The average time for a complete prediction cycle was measured, and system responsiveness and stability were assessed under typical user loads, simulating multiple concurrent users.

Security Testing: Tests included attempts at SQL injection on login and registration forms, checks for Cross-Site Scripting (XSS) vulnerabilities, and verification that CSRF protection is active for all POST requests to prevent unauthorized actions.

Error Handling: The system's response to internal failures, such as preprocessing errors or model loading issues, was tested to ensure appropriate, user-friendly error messages guide the user.

Browser Compatibility: The application was tested across major web browsers—including Chrome, Firefox, and Edge—to ensure consistent behavior, functionality, and responsive design.

System testing confirmed that the DepHNN Analyzer operates reliably as a fully integrated system, maintaining accuracy, robustness, security, and usability in real-world conditions. The consistency of results across multiple test cycles demonstrates that all interconnected modules from authentication to EEG processing and model inference—function cohesively without failures.

7.2.5 ACCEPTANCE TESTING

Acceptance testing, or User Acceptance Testing (UAT), represents the final stage of evaluation, assessing the system from the end-user's perspective to ensure it meets all intended requirements and is ready for deployment. For the DepHNN Analyzer, UAT focused on validating functionality, usability, reliability, and compliance with project objectives.

User Registration and Profile Management: Tests confirmed successful registration, login, and logout for both new and existing users, and verified that OTPs are correctly received and usable for account activation.

EEG File Upload and Analysis: Tests validated the ability to upload various valid .edf EEG files and ensured that analysis completes within acceptable timeframes, maintaining responsiveness.

Prediction Accuracy and Interpretation: The system was tested to provide plausible and consistent predictions, with results—including prediction status, confidence scores, charts, and detailed metrics—clearly displayed and easy to interpret.

Overall System Reliability: UAT confirmed system stability and responsiveness during extended use and verified that all initial project requirements and objectives were fulfilled.

Documentation Review: User-facing documentation, including instructions and guides, was verified for accuracy, clarity, and usefulness.

Acceptance testing confirmed that the DepHNN Analyzer is functionally complete, user-friendly, reliable, and ready for real-world deployment, ensuring that it meets both technical and user requirements.

7.2.6 DATA FLOW TESTING

Data Flow Testing validated the integrity of EEG data as it transforms from a raw .edf file into a final prediction, ensuring that no data is corrupted, misaligned, or lost during preprocessing and prediction.

Upload: Users upload a .edf file via the browser, which the Django backend receives as an UploadedFile object.

Temporary Storage: Files are temporarily saved on the server (e.g., temp_upload_file.edf). Tests confirmed that the saved file matches the uploaded file exactly.

Preprocessing (MNE-Python): The preprocess_single_file function was validated to perform all transformations correctly:

Renaming: Channel names mapped correctly to model-required names.

Filtering & ICA: Artifact-heavy components removed.

Normalization & Windowing: Data segmented into a 3D NumPy array with shape (Number of windows, 19, 256).

Model Input: The resulting array was verified as compatible with the Keras model.predict() method.

Prediction Output: The model produced a 2D array of probabilities, which was traced and verified.

Aggregation & JSON Response: Probabilities were aggregated via majority voting to generate the final prediction and confidence score. The JSON response was sent to the frontend, and tests confirmed correct rendering of results, charts, and metrics.

This end-to-end tracing confirmed that the DepHNN Analyzer maintains data integrity throughout the entire workflow, from raw EEG input to final user-facing prediction.

7.2.7 CONTROL FLOW TESTING

Control Flow Testing was conducted to verify the logical paths and conditional branching within the DepHNN Analyzer Django backend, ensuring that the system behaves correctly and securely based on user status and input data.

Authentication Branching: Templates such as home.html, about.html, and contact.html utilize {% if user.is_authenticated %}. Tests confirmed that logged-in users see the analyzer interface and video background, while logged-out users are presented with the landing page containing login/register options.

Request Method Branching: Views handling form submissions (e.g., register, otp_verify, predict) were tested to confirm that GET requests render forms correctly and POST requests trigger the appropriate processing logic.

OTP Verification Logic: The otp_verify view was tested for conditional branches. Correct OTP submissions activate the user (user.is_active = True) and log them in, whereas incorrect submissions display an error and re-render the OTP form, preventing account activation.

Prediction Logic: The predict view was tested to ensure confidence-based label assignment functions correctly. Confidence thresholds were validated to produce accurate labels: "Major Depressive Disorder," "Depressed," or "Healthy" in the final JSON response.

7.2.8 WHITE BOX TESTING

White box testing was performed to verify the internal structure, logic, and data handling of the Depression Detection System using CNN–LSTM with EEG signals. It ensured that each backend component—EEG preprocessing, feature extraction, and model inference—operated accurately and efficiently. The control flow was tested by executing all conditional statements and loops responsible for EEG file validation, sampling rate verification, and data segmentation. Intermediate data transformations and transitions between different stages of the processing pipeline were validated to confirm that every step executed as designed.

Internal data flow testing was also conducted to trace EEG inputs through the complete pipeline—from file upload to preprocessing steps like noise filtering, normalization, and segmentation. The conversion of EEG signals into NumPy arrays compatible with the CNN–LSTM model was verified for correctness. Each layer of the CNN–LSTM architecture, including convolutional and LSTM layers, was inspected to ensure accurate activation propagation, gradient flow, and learning consistency. The final softmax layer was tested for numerical stability and accurate probability computation during prediction.

This testing confirmed that the internal system logic was sound, with no data loss, misalignment, or unexpected exceptions. Each module performed as intended, maintaining the model's reliability and ensuring that the deep learning pipeline produced consistent, reproducible, and high-quality results during both training and inference.

7.2.9 BLACK BOX TESTING

Black box testing, also known as behavioral testing, focuses on verifying system functionality without insight into internal code or architecture. It validates whether the system

behaves as expected based on input and output. For this Application, black box testing was conducted primarily through the web interface and API endpoints to ensure a seamless user experience and accurate EEG-based depression prediction.

Functional Testing: The main functional test verified that a user could upload a valid EEG file through the interface and receive a classification result such as “Depressed” or “Normal.” Test cases included uploading EEG files with known labels (from the Figshare dataset) and checking the output diagnosis for accuracy. Error-handling scenarios were tested by uploading invalid file types (e.g., .txt or .csv) to confirm that the system correctly displayed an error message and prevented model execution. Additionally, the end-to-end flow — from file upload to preprocessing, model prediction, and result display — was validated to ensure consistency.

User Interface and Usability Testing: The interface was tested for clarity, responsiveness, and usability. Each button and link (e.g., “Upload EEG,” “Predict,” “Results”) was tested for proper function. Progress indicators and loading animations were verified during prediction. The prediction result section was checked to ensure the classification label and confidence percentage were displayed clearly and accurately.

Security and Access Control Testing: Tests were conducted to ensure that only authenticated users could access the prediction module. Attempts to call the /predict/ API without authentication were rejected with an appropriate error message. All data transmission between frontend and backend was verified to occur over secure channels. Invalid EEG uploads or excessive file sizes were also tested to confirm the system could gracefully handle edge cases.

Error Handling Testing: Invalid input formats, missing channels, or corrupted EEG files were tested to ensure the system returned meaningful error messages instead of crashing. Additionally, tests verified that the model’s output remained stable even when encountering slightly noisy or incomplete EEG signals, ensuring robustness.

Through these black box tests, the depression detection system demonstrated high reliability, user-friendliness, and strong protection against unauthorized access or faulty inputs.

7.3 TEST CASES

UNIT TEST CASE

Test Case#	UTC01
Test Name	CustomUser Model Creation
Test Description	To verify creation of a CustomUser with valid data
Input	Valid username, email, password
Expected Output	User is successfully created in the database
Actual Output	User created successfully
Test Result	Success

TABLE 7.1: UNIT TEST CASE - USER MODEL

Test Case#	UTC02
Test Name	OTP Generation Verification
Test Description	Verify that generate_otp() produces a 6-digit OTP and is stored correctly
Input	Trigger OTP generation for a user
Expected Output	OTP generated and saved in the database with correct expiry time
Actual Output	OTP generated and stored correctly
Test Result	Success

TABLE 7.2: UNIT TEST CASE - OTP GENERATION

Test Case#	UTC03
Test Name	RegistrationForm Validation
Test Description	Verify RegistrationForm validates email, password strength, and username uniqueness
Input	New user data with valid and invalid fields

Test Case#	UTC03
Expected Output	Form validates correctly; errors for invalid inputs
Actual Output	Form validated and errors displayed correctly
Test Result	Success

TABLE 7.3: UNIT TEST CASE - DJANGO FORMS**INTEGRATION TEST CASE**

Test Case#	ITC01
Test Name	Registration and OTP Flow
Test Description	Test registration, CustomUser creation, OTP generation, and activation
Input	Registration form submission with valid data
Expected Output	User created, OTP generated, account activated upon OTP verification
Actual Output	Flow executed successfully
Test Result	Success

TABLE 7.4: INTEGRATION TEST CASE - USER REGISTRATION FLOW

Test Case#	ITC02
Test Name	Preprocessing to Prediction Integration
Test Description	Verify preprocess_single_file output correctly feeds model.predict()
Input	Upload valid EEG .edf file
Expected Output	Data passed through preprocessing and prediction returns JSON output
Actual Output	Prediction returned correctly
Test Result	Success

TABLE 7.5: INTEGRATION TEST CASE - PREDICTION FLOW

FUNCTIONAL TEST CASE

Test Case#	FTC01
Test Name	EEG File Upload
Test Description	Verify user can upload a valid EEG file and receive prediction
Input	Valid EEG .edf file
Expected Output	System processes file and displays prediction and confidence score
Actual Output	Prediction and metrics displayed correctly
Test Result	Success

TABLE 7.6: FUNCTIONAL TEST CASE - FILE UPLOAD & ANALYSIS**SYSTEM TEST CASE**

Test Case#	STC01
Test Name	Full Prediction Cycle
Test Description	Execute end-to-end workflow: register, login, upload .edf, predict, view results, logout
Input	Valid EEG files from both healthy and depressed subjects
Expected Output	Predictions returned correctly, system remains stable
Actual Output	Full cycle executed successfully
Test Result	Success

TABLE 7.7: SYSTEM TEST CASE - END-TO-END PREDICTION**WHITE BOX TEST CASE**

Test Case#	WBTC01
Test Name	Authentication Branch Testing

Test Case#	WBTC01
Test Description	Verify {% if user.is_authenticated %} logic in templates and views
Input	Logged-in and logged-out users accessing secure and public pages
Expected Output	Correct content displayed based on authentication status
Actual Output	Displayed correctly for both cases
Test Result	Success

TABLE 7.8: WHITE BOX TEST CASE - CONTROL FLOW

BLACK BOX TEST CASE

Test Case#	BBTC01
Test Name	User Interface Validation
Test Description	Verify all buttons, forms, and links are clickable and functional across devices
Input	User interactions on web UI
Expected Output	UI elements respond correctly, pages load as expected
Actual Output	UI functional on tested devices
Test Result	Success

TABLE 7.9: BLACK BOX TEST CASE - UI TESTING

ACCEPTANCE TEST CASE

Test Case#	ATC01
Test Name	Prediction Accuracy Validation
Test Description	Confirm system provides consistent and correct predictions for known EEG files
Input	Known healthy and depressed EEG files

Test Case#	ATC01
Expected Output	System predicts correctly with clear results and confidence scores
Actual Output	Predictions returned accurately
Test Result	Success

TABLE 7.10: ACCEPTANCE TEST CASE - PREDICTION ACCURACY

CHAPTER 8

CONCLUSION AND FUTURE ENCHANCEMENT